Unlike us humans who live in what appears to be a continuous flow of time, the algorithm lives in a discrete world, so something needs to happen at each discrete time step. At time step 1 the algorithm will "look" at the game board and make a decision about what action to take. Then the game board will be updated, and so on.

Let's sketch out the details of this process now. Here's the sequence of events for a game of Gridworld.

1. We start the game in some state that we'll call S_t. The state includes all the information about the game that we have. For our Gridworld example, the game state is represented as a $4 \times 4 \times 4$ tensor. We will go into more detail about the specifics of the board when we implement the algorithm.
2. We feed the S_t data and a candidate action into a deep neural network (or some other fancy machine-learning algorithm) and it produces a prediction of how valuable taking that action in that state is (see figure 3.2).

   Figure 3.2. The Q function could be any function that accepts a state and action and returns the value (expected rewards) of taking that action given that state.

   

   Remember, the algorithm is not predicting the reward we will get after taking a particular action; it's predicting the expected value (the expected rewards), which is the long-term average reward we will get from taking an action in a state and then continuing to behave according to our policy π. We do this for several (perhaps all) possible actions we could take in this state.
3. We take an action, perhaps because our neural network predicted it is the highest value action or perhaps we take a random action. We'll label the action A_t. We are now in a new state of the game, which we'll call S_{t+1}, and we receive or observe a reward, labelled R_{t+1}. We want to update our learning algorithm to reflect the actual reward we received, after taking the action it predicted was the best. Perhaps we got a